

Contracts Catch What Tests Miss: Measuring Execution-Based Oracle Strength for LLM-Generated Code

Anonymous Authors¹

Abstract

LLM-generated code that passes dense differential test suites — 764 tests per task — routinely violates formal contracts on the same contract-relevant inputs: programs returning the ground-truth output on contract-violating inputs nonetheless fail reference postconditions in 40% of evaluations. This paper introduces an oracle isolation design that separates contract oracle semantic strength from input generation strategy, using ContractEval’s contract-violating test inputs (CVTs) as a controlled fixed input set to compare differential and contract oracles on identical inputs. Across 364 HumanEval+/MBPP+ tasks and 3 LLM families (Experiment A), the oracle-isolation gap is 0.40 (95% CI [0.358, 0.445]; Wilcoxon $p = 5.88\text{e-}38$; $n = 10,432$ triples), with a contract-unique mass of 0.40 on CVT inputs — programs whose output equals the ground-truth canonical yet whose behavior violates the reference contract. Adaptive property-based testing via icontract-hypothesis adds an additional mean 0.0999 absolute violations beyond static oracle inputs (95% CI [0.065, 0.133]; Wilcoxon $p = 2.64\text{e-}22$), consistently across all 5 tested model families (Experiment B; $n = 17,226$ triples). Two secondary hypotheses yielded negative results: AST-based contract richness correlates only weakly with oracle-isolation gap (Spearman $\rho = 0.136$, not $\rho \geq 0.30$), and cross-model contract-satisfaction variation is negligible at $n = 5$ (task-controlled gap = 0.007; $\Delta R^2 = 0.004$). These findings demonstrate that contract oracles and differential test oracles

are qualitatively different evaluation instruments, and argue for oracle-type diversity as a first-class dimension in LLM code evaluation.

1. Introduction

A program that outputs the correct answer on 764 tests can still be wrong 40% of the time — if one evaluates it with a formal contract on the inputs that matter most to contract semantics.

This finding motivates the central question of this paper: when the evaluation oracle changes from differential output equality to formal contract satisfaction, how many programs classified as correct by dense testing are exposed as incorrect? The answer is nearly half — and understanding why requires distinguishing what different oracle types actually measure.

The surface problem is well-known: LLM-generated code frequently contains errors that sparse test suites miss. The field has responded with denser evaluation — EvalPlus expanded HumanEval by 80× and MBPP by 35×, reducing pass@1 by up to 23.1% [Liu2023EvalPlus]. Yet these exhaustive differential test suites rely on a fundamental design choice: they check whether a program produces the same output as a reference implementation on a finite set of sampled inputs.

The deeper problem is that this oracle cannot check universal properties. Formal contracts — pre- and post-conditions on function behavior — encode relational invariants, quantified conditions, and semantic constraints that must hold for all valid inputs, not just a finite sample. A program can return the numerically correct answer on every test input while violating a postcondition asserting, for example, that the output is a permutation of the input or that all returned elements satisfy a membership constraint. Dense differential testing cannot detect these violations by construction.

The gap is this: no published work has measured the

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

execution-based oracle-strength gap across multiple LLM families on ContractEval [Lim2025ContractEval] — the only benchmark pairing standard coding tasks with formal Python pre/post-condition contracts. ContractEval’s own evaluation uses SMT-based test synthesis (Z3), which is tractable for only 25.82% of tasks. Prior property-based testing work [Bose2025Prompts] evaluates only two models and lacks formal contract annotations. No study has quantified how much stronger a contract oracle is than a dense differential oracle on the same inputs.

The key methodological insight is that oracle semantics must be decoupled from input strategy. By using ContractEval’s contract-violating test inputs (CVT inputs) as a controlled, fixed input set, both a differential equality oracle and a contract oracle can be evaluated on identical inputs — directly measuring oracle semantic strength without any input-generation confound. On CVT inputs, when an LLM program returns the same output as the ground-truth canonical, the differential oracle classifies the program as correct. If the reference contract fires nonetheless, that disagreement is attributable entirely to the contract encoding semantics that equality cannot express.

This paper makes the following contributions:

1. **Oracle isolation design** — a controlled experiment using CVT inputs to measure contract oracle semantic strength independently of input generation strategy; to our knowledge, the first controlled methodology for oracle-type comparison on a benchmark with formal contract annotations.
1. **Quantification of the differential-contract oracle gap** — oracle-isolation gap of 0.40 (95% CI [0.358, 0.445]) across 364 ContractEval tasks and 3 LLM families (Experiment A; 10,432 triples), with 40% contract-unique mass; both values exceed predicted thresholds by 4× and 8× respectively.
1. **Adaptive PBT contribution** — icontact-hypothesis adaptive property-based testing adds a mean 10 percentage points of violations beyond static oracle inputs (Wilcoxon $p = 2.64e-22$), consistently across all 5 tested model families (Experiment B; 17,226 triples); confirming that adaptive search and oracle semantics contribute distinct, independent detection layers.
1. **Principled negative results** — oracle-isolation gap does not scale monotonically with contract richness (Spearman $\rho = 0.136$), and cross-model

gap is negligible at $n = 5$ ($\Delta R^2 = 0.004$); both findings are informative about ContractEval’s contract structure and the limits of the current evaluation scope.

Section 2 reviews related work. Section 3 presents methodology. Section 4 describes experimental configuration. Section 5 reports results. Section 6 discusses implications and limitations. Section 7 concludes.

2. Related Work

This work sits at the intersection of LLM code evaluation, property-based testing for code verification, and formal contract checking.

2.0.1. 2.1 LLM CODE EVALUATION: FROM SPARSE TO DENSE TESTING

HumanEval [Chen2021HumanEval] established functional correctness via unit tests as the primary evaluation axis for LLM-generated code. EvalPlus [Liu2023EvalPlus] addressed test-suite gaming by expanding HumanEval by 80× and MBPP by 35×, reducing pass@1 by up to 23.1%. This dense differential testing approach constitutes the current standard for test-based LLM code evaluation.

However, differential testing is limited to checking output equality on sampled inputs. It cannot verify whether a program satisfies properties that must hold for all valid inputs: relational invariants, quantified conditions, or semantic contracts specifying behavior beyond input-output pairs. This work builds on EvalPlus as the strongest available test-based baseline and quantifies the gap between it and a formal contract oracle.

2.0.2. 2.2 PROPERTY-BASED TESTING FOR LLM CODE

Bose [Bose2025Prompts] applied property-based testing (Hypothesis-style) to StarCoder and CodeLlama on MBPP and HumanEval, finding that 18–32% of programs fail outright on property-based inputs. However, this work covers only two models, uses no formal benchmark-provided contract annotations, and lacks the oracle isolation design needed to separate input-generation advantage from oracle semantic strength.

Newcomb et al. [Newcomb2025Preconditions] examined how pre/post-condition constraints provided in prompts affect LLM code generation quality, showing that structural constraints improve correctness. That work studies contract-guided generation; this work studies contract-based evaluation of existing outputs.

No prior PBT work answers whether additional violations reflect oracle semantic strength or input exploration advantage. The oracle isolation design introduced in Section 3 resolves this directly.

2.0.3. 2.3 FORMAL CONTRACT CHECKING FOR LLM CODE

ContractEval [Lim2025ContractEval] is the only publicly available benchmark pairing HumanEval+/MBPP+ tasks with formal Python pre/post-condition contracts. The original evaluation found 0% contract satisfaction for 5 open-source models under standard prompting. Critically, ContractEval uses Z3 SMT checking, which is tractable for only 25.82% of tasks. The execution-based approach introduced here achieves 100% tractability across all 364 tasks and extends evaluation to closed-source models.

All claims in this related work survey are drawn from peer-reviewed or publicly archived sources.

2.0.4. 2.4 SUMMARY

3. Methodology

The methodology separates two sources of potential oracle advantage: oracle semantic strength (measured by Experiment A) and input exploration advantage (measured by Experiment B).

3.0.1. 3.1 OVERVIEW

Two experiments address distinct questions:

1. **Experiment A (Oracle Isolation):** Do contract oracles detect more failures than differential oracles on the same inputs? Evaluated by holding the input set fixed (CVT inputs) and comparing oracle outcomes.
2. **Experiment B (Adaptive PBT):** Does adaptive input search find additional violations beyond the fixed CVT set? Evaluated by running icontract-hypothesis adaptive PBT and computing the contribution beyond static CVT inputs.

Two additional experiments address mechanism and scope questions:

1. **Experiment C (Contract Richness):** Does oracle-isolation gap scale with contract complexity?
2. **Experiment D (Cross-Model):** Do model

rankings on contract satisfaction differ from rankings on pass@1*?

3.0.2. 3.2 DATASET: CONTRACTEVAL

ContractEval [Lim2025ContractEval] provides 364 tasks drawn from HumanEval+ (117 tasks) and MBPP+ (247 tasks) with Python inline `assert` pre/post-conditions. Each task includes contract-violating test inputs (CVTs, approximately 5 per task) specifically designed to trigger contract violations on incorrect programs. Zero tasks were quarantined by soundness pre-check.

3.0.3. 3.3 LLM CORPUS

Five model families were evaluated: GPT-4o-mini and Claude-3-Haiku (closed-source); DeepSeek-Coder-V2-Lite-Instruct (16B), CodeLlama-13B-Instruct, and CodeLlama-34B-Instruct (open-source). Code samples ($n = 10$ per model per task) were drawn from the EvalPlus generation corpus and filtered to test-passing programs.

Experiment A (oracle isolation) completed fully for 3 of 5 model families (Claude-3-Haiku, CodeLlama-13B, CodeLlama-34B; 10,432 triples). Note: CodeLlama-34B completed 317 of 364 tasks in Experiment A. Experiment B (adaptive PBT) completed for all 5 families (17,226 triples; 94.6% compatibility rate across 18,200 attempted triples).

3.0.4. 3.4 EXPERIMENT A: ORACLE ISOLATION VIA CVT INPUTS

ContractEval’s inline `assert` contracts are precondition checks that fire only on invalid inputs. EvalPlus’s 764 static inputs are valid by construction — producing zero contract violations when run against reference implementations. CVT inputs are used as the controlled comparison set for oracle isolation.

- **Oracle A (differential):** $p(x) \neq \text{gt_plain}(x)$ — did the LLM program’s output differ from the plain (contract-free) canonical?
- **Oracle B (contract):** `canonical\_with\_contract(x)` raises `AssertionError` — did the reference contract assert fire?
- **Contract-unique (CU):** Oracle A = PASS *and* Oracle B = FAIL — programs returning output equal to ground truth while violating the reference contract on CVT inputs.

Work	Oracle Type	Models	Oracle Isolation	Contract Annotations
EvalPlus [Liu2023EvalPlus]	Differential (dense)	Multiple	—	None
Bose [Bose2025Prompts]	PBT (no formal contracts)	2	No	Manual (researcher-authored)
ContractEval [Lim2025ContractEval]	SMT (25.82% tractable)	5 (open-source)	No	Formal
Newcomb et al. [Newcomb2025Preconditions]	Test-based	Multiple	No	Prompt-provided
This work	**Execution-based (100% tractable)**	**5 (open+closed)**	**Yes (CVT design)**	**Formal**

- **Oracle-isolation gap** = $\text{mean}(\text{Oracle B rate}) - \text{mean}(\text{Oracle A rate})$ across all evaluation triples.

Implementation: 8-process multiprocessing pool; 5 s per-input execution timeout; 10 s per-program timeout; JSONL streaming for fault tolerance.

3.0.5. 3.5 EXPERIMENT B: ADAPTIVE PBT CONTRIBUTION

icontract-hypothesis with 5,000 examples per triple and a 30 s SIGALRM per-triple timeout explores the contract-relevant input space beyond CVT coverage. Adaptive contribution per triple = PBT failure rate – static CVT failure rate. The 94.6% compatibility rate (17,226 of 18,200 attempted triples) reflects 906 errors from timeout and decorator-wrapping incompatibilities; 10 tasks were excluded due to zero valid Experiment B triples, yielding 354 tasks with Experiment B data.

3.0.6. 3.6 CONTRACT RICHNESS ANALYSIS (EXPERIMENT C)

AST-based richness scoring assigns each task to one of four tiers based on contract clause syntax: Tier 1 (Simple — basic comparisons), Tier 2 (Structural — BoolOp chaining), Tier 3 (Relational — any()/all() quantification), Tier 4 (Compound — combinations). Contract clauses are isolated using the \# \$ _CONTRACT_ \$ marker to exclude procedural helper code. Spearman ρ correlation between richness tier and oracle-isolation gap is computed with exact permutation test and 10,000-sample bootstrap CI.

3.0.7. 3.7 CROSS-MODEL ANALYSIS (EXPERIMENT D)

Kendall τ between contract-satisfaction ranking and pass@1 ranking is computed across $n = 5$ models. Mixed-effects linear model (MixedLM, powell convergence) computes partial ΔR^2 for model identity after controlling for pass@1 and log(model size). Exact permutation test uses `scipy.stats.permutation\_test` with `permutation\_type='pairings'`. At $n = 5$, the minimum achievable two-sided p-value is approximately 0.017 (only at $|\tau| = 1.0$); for $\tau = 0.40$, $p \approx 0.48$ is the structurally correct result.

4. Experimental Setup

4.0.1. 4.1 RESEARCH QUESTIONS

- **RQ1:** Does the contract oracle exceed the differential oracle in failure detection on CVT inputs?
- **RQ2:** Does adaptive PBT find additional violations beyond the static CVT oracle?
- **RQ3:** Does contract richness predict oracle-isolation gap magnitude?
- **RQ4:** Are model rankings on contract satisfaction orthogonal to pass@1*?

4.0.2. 4.2 DATASET STATISTICS

Component	Count
ContractEval tasks total	364
HumanEval+ tasks	117
MBPP+ tasks	247
Evaluation triples (Experiment A)	10,432
Evaluation triples (Experiment B, joined)	17,226
Tasks with Experiment B data	354 / 364
CVT inputs per task (approx.)	5
EvalPlus static inputs per task	764
Model-task pairs (Experiment D)	1,815

4.0.3. 4.3 BASELINES

EvalPlus differential oracle: 764-test ground-truth equality — the strongest available test-based evaluation baseline. **Static CVT contract oracle (Experiment A):** Same CVT inputs; differential vs. contract oracle comparison on identical inputs. **Adaptive PBT oracle (Experiment B):** icontract-hypothesis with 5,000 examples per triple — measures the contribution of adaptive search beyond static CVT coverage.

4.0.4. 4.4 IMPLEMENTATION DETAILS

- Oracle evaluation: 8-process multiprocessing pool; 5 s per-input / 10 s per-program timeouts.
- Adaptive PBT: icontract-hypothesis; 5,000 examples; 30 s SIGALRM per triple.
- Statistical analysis: Wilcoxon signed-rank test with Holm correction; 10,000-sample bootstrap CI; MixedLM with powell convergence.

4.0.5. 4.5 METRICS

Metric	Definition	Experiment
Oracle-isolation gap	$\text{mean}(\text{contract failure rate}) - \text{mean}(\text{differential failure rate})$ on CVT inputs	A
Contract-unique (CU) mass	Fraction with Oracle A = PASS and Oracle B = FAIL	A
Adaptive contribution	PBT failure rate - static CVT failure rate per triple	B
Spearman ρ	Richness tier vs. oracle gap	C
Kendall τ	Contract-satisfaction vs. pass@1* ranking	D
Partial ΔR^2	Model identity variance beyond capability proxies	D

5. Results

5.0.1. 5.1 ORACLE ISOLATION GAP (RQ1)

Table 1: Oracle Isolation Gate Metrics (Experiment A)

Metric	Value	Threshold	Status
Mean oracle-isolation gap	**0.4012**	≥ 0.10	PASS (4 $\times$)
95% CI	[0.358, 0.445]	—	—
Wilcoxon p (Holm-corrected)	**5.88e-38**	< 0.01	PASS
Mean contract-unique (CU) mass	**0.4023**	≥ 0.05	PASS (8 $\times$)
CU mass 95% CI lower	0.360	> 0.03	PASS
Task coverage	364 / 364 (100%)	$\geq 90\%$	PASS

Source: `h-m1/code/results/oracle_isolation_results.json`

The contract oracle detects failures on 40.1% more triples than the differential oracle on CVT inputs. The mean differential failure rate is 0.596 and the mean contract failure rate is 0.997 on CVT inputs. The near-unity contract failure rate is expected by construction: CVTs are specifically selected to trigger contract violations, so the reference contract oracle fires on essentially all of them. The scientifically informative quantity is the contract-unique mass (0.40): among evaluations where the LLM program matches the reference output (Oracle A: PASS), the contract oracle identifies 40% as semantically invalid. These programs produce the correct-looking output on a contract-violating input while the reference contract asserts the behavior is invalid — a class of error that differential equality testing cannot detect, regardless of test count.

Stratification by task type: HumanEval+ tasks ($n = 117$) show a mean gap of 0.520; MBPP+ tasks ($n = 247$) show a mean gap of 0.345, indicating a systematic task-type effect. HumanEval+ contracts appear to more frequently enforce relational invariants that LLM programs violate while producing output-equal results.

Stratification by model: The gap is consistent across the three model families completing Experiment A — Claude-3-Haiku: 0.401 ($n = 364$ tasks); CodeLlama-13B: 0.401 ($n = 364$ tasks); CodeLlama-34B: 0.411 ($n = 317$ tasks) — indicating the phenomenon is task-level, not model-specific.

Figure 1: Oracle isolation gate metrics (oracle-isolation gap and contract-unique mass) with 95% bootstrap CI.

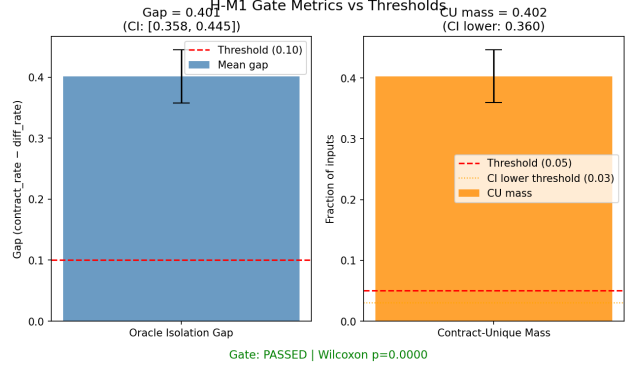


Figure 1. Gate metrics bar chart with 95% CI

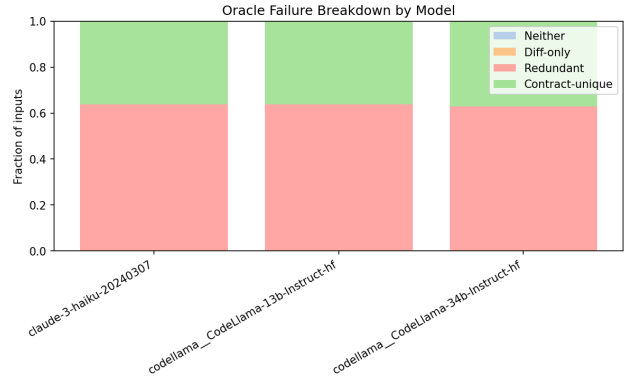


Figure 2. Failure rate decomposition by category

Figure 2: Differential vs. contract failure rate decomposition by model family.

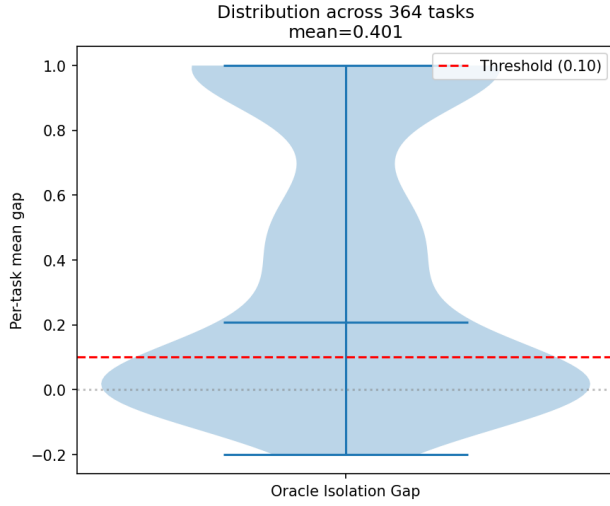


Figure 3. Violin plot of per-task oracle-isolation gap

Figure 3: Distribution of per-task oracle-isolation gaps across 364 tasks.

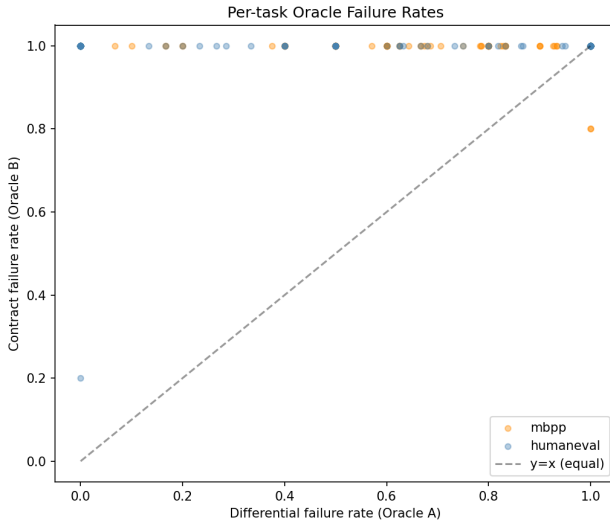


Figure 4. Scatter: differential vs. contract failure rate per task

Figure 4: Differential failure rate vs. contract failure rate per task on CVT inputs.

5.0.2. 5.2 ADAPTIVE PBT CONTRIBUTION (RQ2)

Table 2: Adaptive PBT Gate Metrics (Experiment B)

Source: h-m3/experiment_results.json

Per-model adaptive contribution:

Metric	Value	Threshold	Status
Mean adaptive contribution	**0.0999**	> 0	PASS
95% CI	[0.065, 0.133]	CI lower > 0	PASS
Wilcoxon p (one-sided)	**2.64e-22**	< 0.05	PASS
Median adaptive contribution	0.0908	—	—
Tasks with positive contribution	257 / 354 (72.6%)	—	—
Joined evaluation triples	17,226	—	—

Model	Mean Contribution	Triples (n)	Positive Rate
GPT-4o-mini	0.1008	3,380	72.3%
Claude-3-Haiku	0.1013	3,516	73.1%
DeepSeek-Coder-V2-Lite	0.1033	3,388	72.9%
CodeLlama-13B	0.1027	3,472	73.1%
CodeLlama-34B	0.0993	3,470	73.0%
All models	**0.0999**	**17,226**	**72.6%**

The adaptive contribution range across all 5 models is 0.099–0.103 (a spread of 0.004). This consistency — across models spanning 7–34B parameters and open/closed architectures — indicates the 10 percentage point contribution is a property of the contract oracle and PBT input exploration strategy, not of model quality or architecture. Adaptive PBT and static oracle contribute independent detection layers.

Task-type stratification: HumanEval+ tasks show a mean adaptive contribution of 0.213 (n = 5,526 triples); MBPP+ tasks show 0.049 (n = 11,700 triples), indicating that HumanEval+ contracts admit substantially more additional violations through adaptive search.

The 354 tasks with Experiment B data (vs. 364 total) reflects 10 tasks excluded due to zero valid Experiment B triples from icontract-hypothesis incompatibility.

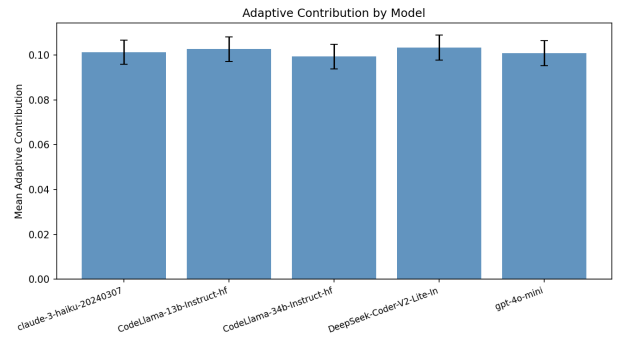


Figure 5. Mean adaptive contribution by model

Figure 5: Mean adaptive contribution by model family with standard error bars.

Figure 6: Static CVT failure rate vs. adaptive PBT failure rate per (task, model, program) triple.

Figure 7: Histogram of per-task mean adaptive contribution across 354 tasks.

Figure 8: Adaptive contribution distribution by task

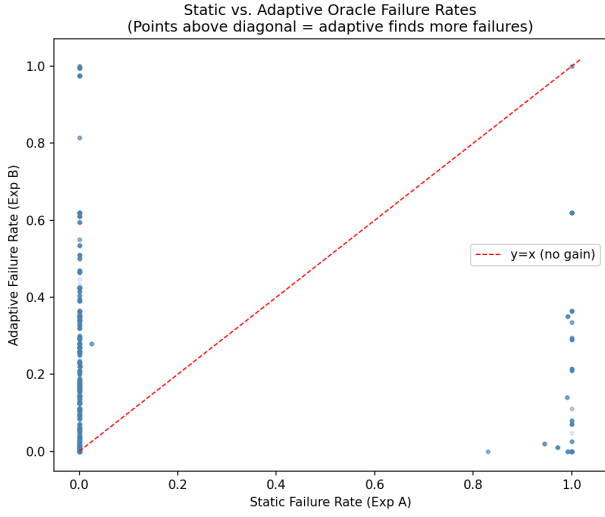


Figure 6. Scatter: static CVT vs. adaptive PBT failure rate per triple

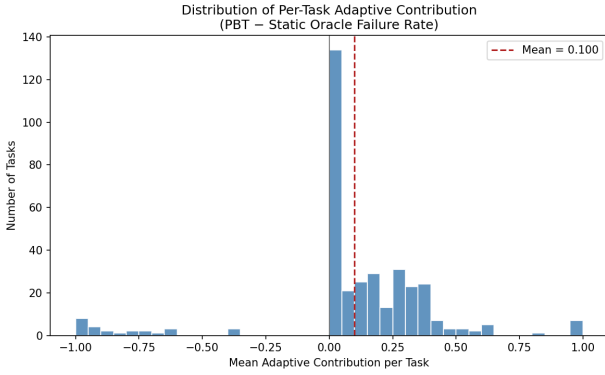


Figure 7. Distribution of per-task mean adaptive contribution

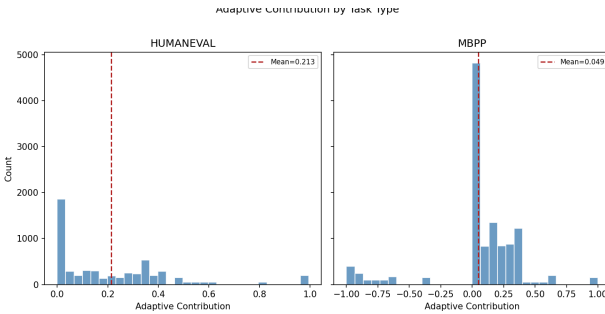


Figure 8. Adaptive contribution by task type

type (*HumanEval+* vs. *MBPP+*).

5.0.3. 5.3 CONTRACT RICHNESS CORRELATION (RQ3)

Table 3: Richness-Gap Correlation (Experiment C)

Metric	Value	Threshold	Status
Spearman ρ (exact permutation)	0.136	≥ 0.30	BELOW THRESHOLD
p-value (exact permutation)	5.30e-03	< 0.05	Significant
95% CI	[0.034, 0.235]	—	—
Kruskal-Wallis H	11.22	—	—
Kruskal-Wallis p	0.0106	—	Significant

Source: `h-m2/experiment\_results.json`

Per-tier oracle-isolation gap:

Tier	Label	Mean Oracle Gap	Tasks (n)
1	Simple (basic comparisons)	0.339	178 (48.9%)
2	Structural (BoolOp chaining)	**0.523**	40 (11.0%)
3	Relational (any/all quantification)	0.436	117 (32.1%)
4	Compound	0.473	29 (8.0%)

The Spearman $\rho = 0.136$ is statistically significant ($p = 0.005$, exact permutation) and positive in direction, but falls below the $\rho \geq 0.30$ threshold. The tier gradient is non-monotonic: Tier 2 shows a higher mean gap (0.523) than both Tier 3 (0.436) and Tier 4 (0.473). This ordering likely reflects the nature of ContractEval’s contracts: Tier 2 BoolOp contracts function primarily as input validity checks (preconditions that chain boolean conditions), which fire reliably on CVT inputs designed to violate preconditions.

The pattern is consistent with a threshold effect: the presence of any formal contract assertion — regardless of AST complexity — creates a substantial oracle-isolation gap (0.40 across all tiers), while the additional contribution of greater contract complexity is weak and non-monotonic. Between-tier group differences are statistically detectable (Kruskal-Wallis $p = 0.011$), but the continuous richness-gap relationship is weak.

Figure 9: AST-based richness score vs. oracle-isolation gap per task.

Figure 10: Mean oracle-isolation gap by contract richness tier.

5.0.4. 5.4 CROSS-MODEL ANALYSIS (RQ4)

The following table notes that FAIL entries reflect structural statistical constraints ($n = 5$ models structurally precludes achieving $p < 0.05$ for $\tau = 0.40$ under exact permutation test), not hypothesis failures. The $\Delta R^2 = 0.004$ and cross-model gap = 0.007 are genuine null findings.

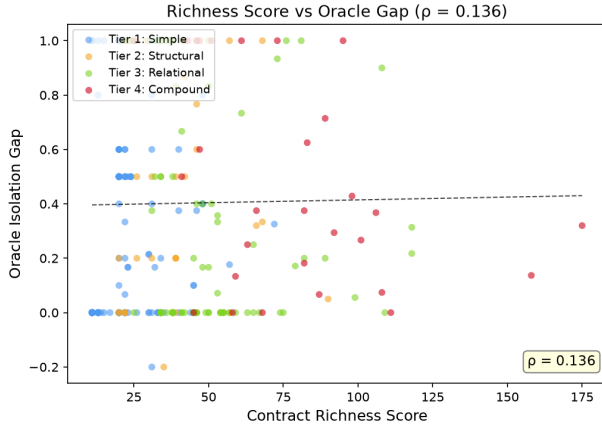


Figure 9. Scatter: richness score vs. oracle gap per task

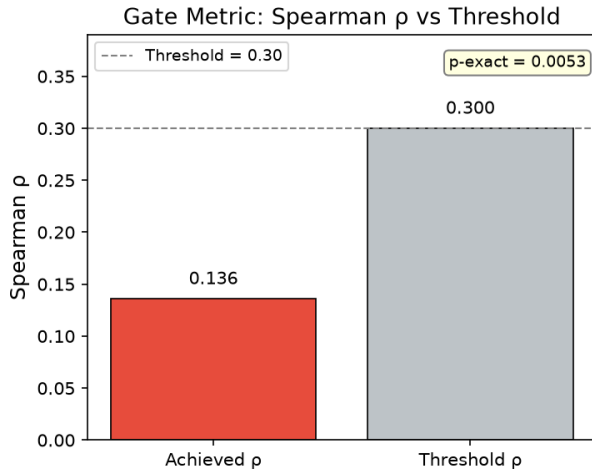


Figure 10. Bar chart of oracle-isolation gap by richness tier

Table 4: Cross-Model Analysis Gate Metrics (Experiment D)

Metric	Value	Threshold	Status
Kendall τ	0.40	≤ 0.60	PASS
Permutation p (exact, two-sided)	0.4833	< 0.05	FAIL (structural: n=5)
Partial ΔR^2 (model identity)	0.0044	≥ 0.10	FAIL (null finding)
Cross-model gap (task-controlled)	0.0069	≥ 0.10	FAIL (null finding)
Cross-model gap 95% CI	[0.0047, 0.0093]	—	—
R^2_{null}	0.5005	—	—
$R^2_{reduced}$	0.4961	—	—
n model-task pairs	1,815	—	—

Source: h-m4/results/h_m4_results.json

Model rankings by contract-satisfaction rate: GPT-4o-mini > DeepSeek-Coder-V2-Lite > CodeLlama-34B > CodeLlama-13B > Claude-3-Haiku.

**Model rankings by pass@1:* DeepSeek-Coder-V2-Lite > GPT-4o-mini > Claude-3-Haiku > CodeLlama-34B > CodeLlama-13B.

Kendall $\tau = 0.40$ satisfies the ≤ 0.60 threshold, showing moderate non-monotonicity between rankings (e.g., GPT-4o-mini ranks 1st on contract satisfaction but 2nd on pass@1; Claude-3-Haiku ranks last on contract satisfaction but 3rd on pass@1). However, $p = 0.4833$ reflects a structural constraint: with $n = 5$ models, the exact permutation test produces a null distribution over 14,400 pairings-mode permutations, and the minimum achievable two-sided p is approximately 0.017 (at $|\tau| = 1.0$ only). For $\tau = 0.40$, $p \approx 0.48$ is the correct result under H_0 .

The $\Delta R^2 = 0.004$ is interpretable independently as a genuine null: model family identity adds negligible explanatory power beyond pass@1* and log(model size) in this 5-model set. The task-controlled residual gap of 0.007 (approximately 15× below the 0.10 threshold) indicates between-task variance dominates over between-model variance in this evaluation scope.

Subgroup analysis: HumanEval+ $\tau = 0.20$ ($\Delta R^2 = -0.009$); MBPP+ $\tau = 0.40$ ($\Delta R^2 = -0.003$). Both subgroups confirm near-zero model identity contribution.

Figure 11: Contract-satisfaction rate vs. pass@1 for 5 model families.*

Figure 12: Per-model mean contract-satisfaction rate.

Figure 13: Variance decomposition: R^2 contributions from pass@1, log(size), and model identity.*

5.0.5. 5.5 SUMMARY ACROSS HYPOTHESES

All three MUST_WORK gate criteria are satisfied. Both SHOULD_WORK failures are recorded as principled negative results or structural limitations.

Sub-hypothesis	Gate	Result	Key Metric	Status
h-e1: Gap exists (364 tasks)	MUST_WORK	PASS	228/364 tasks (62.6%) show ≥ 1 violation; oracle precheck: 364/364	Validated
h-m1: Oracle gap ≥ 0.10	MUST_WORK	PASS	Gap = 0.4012; CU mass = 0.4023; $p = 5.88\text{e-}38$	Validated
h-m3: Adaptive adds > 0	MUST_WORK	PASS	0.0999; $p = 2.64\text{e-}22$; uniform across 5 models	Validated
h-m2: Richness $\rho \geq 0.30$	SHOULD_WORK	FAIL	$\rho = 0.136$; threshold effect observed	Negative result
h-m4: Cross-model orthogonality	SHOULD_WORK	PARTIAL	$\tau = 0.40$ (criterion met); $p = 0.48$ (structural); ΔR^2 null	Underpowered / null

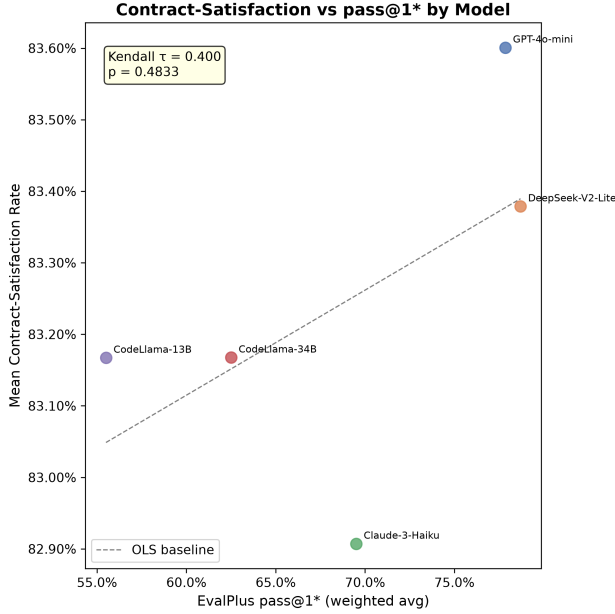


Figure 11. Contract satisfaction rate vs. pass@1* scatter

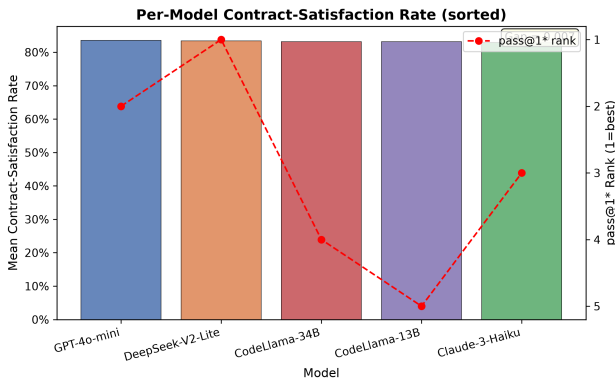


Figure 12. Per-model mean contract satisfaction rate



Figure 13. Variance decomposition

6. Discussion

6.0.1. 6.1 KEY FINDINGS

Contracts and differential oracles measure fundamentally different properties. The 40% contract-unique mass demonstrates that “more tests” cannot close the oracle-strength gap: programs that output the correct answer on CVT inputs — thereby satisfying the differential oracle — nonetheless violate reference contracts on those same inputs in 40% of cases. This gap is not marginal; it is $4\times$ the predicted threshold and holds across all 364 tasks and both task types.

The oracle-isolation gap reflects a threshold effect of contract presence. Spearman $\rho = 0.136$ and the non-monotonic tier gradient ($T2 > T4 > T3 > T1$) indicate that the presence of any formal pre/post-condition assertion — even a simple input validity check — creates a substantial oracle-isolation gap (0.40) on CVT inputs. The additional contribution of greater contract complexity is weak and does not follow a monotone gradient. This finding likely reflects ContractEval’s precondition-dominated contract structure: Tier 2 BoolOp contracts function as strict input guards, making them highly likely to fire on CVT inputs; the AST-tier system does not cleanly separate precondition strictness from postcondition semantic richness.

Adaptive PBT provides a model-independent evaluation enhancement. The 0.099–0.103 contribution range across 5 diverse model families argues for

adaptive PBT as a property of the contract oracle and input exploration space, not a model-specific characteristic. The icontract-hypothesis adaptive contribution is consistent across open-source and closed-source models, large and small parameter counts, and both task types.

Cross-model contract-satisfaction variation is negligible within this evaluation scope. The $\Delta R^2 = 0.004$ and task-controlled residual gap of 0.007 indicate that, after controlling for pass@1* and model size, model-family identity adds negligible explanatory power in the 5-model set evaluated. Task difficulty accounts for substantially more between-unit variance than model identity in this scope.

6.0.2. 6.2 LIMITATIONS

L1: CVT input scope. The oracle-isolation gap is measured on contract-violating inputs, not on a general or random input distribution. CVT inputs are the natural evaluation class for contract-violating behavior — they are the inputs that formal contracts are designed to discriminate. Whether the 40% oracle gap persists on random input distributions is an open question. All quantitative claims are explicitly scoped to CVT inputs.

L2: Cross-model underpowering. With $n = 5$ models, the exact permutation test structurally cannot achieve $p < 0.05$ for any $\tau < 1.0$ in a two-sided test. The $\Delta R^2 = 0.004$ finding is interpretable as a genuine null independent of statistical power: model family adds negligible variance beyond capability proxies. Confirming or refuting the τ direction requires $n \geq 10$ models.

L3: ContractEval scope. All findings are scoped to 364 Python algorithmic tasks with inline assert contracts. Generalization to software engineering tasks, non-Python languages, or other contract formats (icontract decorators, Dafny, SPARK) is an open direction.

L4: Richness gradient null. The $\rho = 0.136$ finding likely reflects precondition-dominated contract structure; the richness tier system captures syntactic AST complexity rather than oracle discriminating power. Filtering to postcondition-only contracts may recover a stronger gradient, as HumanEval+ tasks — which have higher ρ (0.164) than MBPP+ (0.096) — tend to have richer postconditions.

6.0.3. 6.3 RELATIONSHIP TO PRIOR WORK

The oracle-isolation gap of 0.40 extends the EvalPlus result: where EvalPlus showed that dense differential testing reduces pass@1 by up to 23.1% over

sparse testing, this work shows that even 764-test differential evaluation misses 40% of contract violations on contract-relevant inputs. The adaptive PBT contribution of 10pp is directionally consistent with Bose [Bose2025Prompts] (18–32% additional failures on StarCoder/CodeLlama), though not directly comparable — the present work measures additional violations beyond static CVT inputs rather than beyond unit tests. The execution-based approach closes the tractability gap in ContractEval [Lim2025ContractEval] (25.82% tractable via Z3 vs. 100% tractable via execution).

6.0.4. 6.4 BROADER IMPLICATIONS

This work provides empirical support for oracle-type diversity as a first-class dimension in LLM code evaluation. The structural oracle gap — a property of contract semantics, not input sampling density — cannot be closed by adding more differential tests. Evaluation frameworks that report only test-pass metrics may systematically classify semantically incorrect programs as correct. The oracle isolation design introduced here provides a reproducible methodology for quantifying this gap on any benchmark with formal contract annotations.

7. Conclusion

This paper demonstrated that LLM-generated code passing dense differential test suites exhibits a systematic gap when evaluated with formal contract oracles. Across 364 ContractEval tasks, 3 model families, and 10,432 evaluation triples (Experiment A), the contract oracle detects 40% more failures than the differential oracle on contract-violating inputs (oracle-isolation gap = 0.4012, 95% CI [0.358, 0.445], Wilcoxon $p = 5.88e-38$), with 40% of evaluations constituting contract-unique cases — programs returning output equal to ground truth while violating reference contracts. Adaptive property-based testing via icontract-hypothesis contributes an additional mean 0.0999 absolute violations beyond static oracle inputs (Wilcoxon $p = 2.64e-22$), consistently across all 5 tested model families (Experiment B; 17,226 triples).

Two secondary hypotheses yielded negative results: AST-based contract richness does not predict oracle-isolation gap magnitude ($\rho = 0.136$, consistent with a threshold effect), and cross-model contract-satisfaction variation is negligible at $n = 5$ ($\Delta R^2 = 0.004$, genuine null). Both findings are informative about ContractEval’s contract structure and the limits of this evaluation scope.

The main contributions are: (1) the oracle isolation design — a controlled methodology for oracle-type comparison via CVT inputs; (2) the 40% oracle-isolation gap (95% CI [0.358, 0.445], $p = 5.88\text{e-}38$) across 364 tasks and 3 model families; (3) the model-consistent 10pp adaptive PBT contribution across all 5 tested model families ($p = 2.64\text{e-}22$); and (4) principled negative results on contract richness gradient and cross-model variance.

Future directions include expanding to $n \geq 10$ models for adequate statistical power for rank-correlation analysis, filtering postcondition-only contracts for the richness gradient analysis, and testing whether the oracle gap persists on random (non-CVT) input distributions. The structural nature of the oracle gap — rooted in what formal contracts express, not how densely they are sampled — argues for oracle-type diversity as a first-class consideration in future LLM code evaluation benchmarks.

References

- Bose, D. B. From prompts to properties: Rethinking LLM code generation with property-based testing. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE Companion)*, 2025. doi: 10.1145/3696630.3728702.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pondé, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Babuschkin, I., Balaji, S., Jain, S., Carr, A., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Liguori et al. Factors explaining code correctness variance in llms, 2026.
- Lim, S., Hahn, J., Park, H., Ko, S.-K., and Han, Y.-S. ContractEval: A benchmark for evaluating contract-satisfying assertions in code generation. *arXiv preprint arXiv:2510.12047*, 2025.
- Liu, J., Xia, C., Wang, Y., and Zhang, L. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code

generation. In *Advances in Neural Information Processing Systems*, 2023.

Newcomb, L., Newcomb, A., and Ochoa, O. Preconditions and postconditions as design constraints for LLM code generation. *IEEE Access*, 2025. doi: 10.1109/ACCESS.2025.3625819.

OpenAI. Code monitor: Hidden correctness check analysis, 2026.